RAG Assignment Report

Baseline Retrieval vs. Cross-Encoder Re-Ranking

Course: LMA
Roll Number: 2025202013
Name: Anurag Kaushal
Repository: github.com/Anurag-1205/rag_assign1

In this assignment I compare a standard dense-retrieval baseline against a two-stage pipeline that adds cross-encoder re-ranking on top of the same retrieved candidates. My aim is to examine not only whether re-ranking improves answer quality on adversarial queries, but also to understand where it fails and what the latency trade-off looks like in practice.

1. Introduction to Dataset

I chose the **Digital Personal Data Protection Act, 2023** together with the **Digital Personal Data Protection Rules, 2025** as the corpus. These two documents were merged into a single 39-page PDF. Legal text is a practical stress-test for retrieval systems: the answer to a question is often a short qualification hidden beside a broader rule (an exception, a prohibition, or a role-specific amount), while large portions of surrounding text discuss very similar topics using overlapping vocabulary.

After cleaning the raw PDF, I split the text into **230 chunks of 120 words each** with a 30-word overlap between adjacent chunks. The overlap ensures that a key clause sitting on a chunk boundary is not split across two retrieval units. All experiments below use this fixed set of 230 chunks; nothing about the corpus changes between pipelines.

Property	Value
Source documents	DPDPA 2023 + DPDP Rules 2025
Total pages	39
Number of chunks	230
Chunk size / overlap	120 words / 30 words
Evaluation pool	112 adversarial queries (Distractor 28, Exception 73, Negation 11)

Table 1. Corpus and evaluation pool statistics.

2. Architecture Comparison

The two pipelines share an identical corpus, retrieval budget (top-10 candidates per query), and LLM call. The only difference is what happens between the initial retrieval and the LLM prompt.

2.1 Bi-Encoder (Pipeline 1, Baseline)

A bi-encoder processes the query and each document chunk *independently*. Both are mapped to fixed-length vectors, and similarity is measured as the cosine between them. Because chunk representations are pre-computed and stored, inference requires only one forward pass to embed the query, followed by a fast nearest-neighbour search. I used **all-MiniLM-L6-v2** with L2-normalised embeddings and a FAISS IndexFlatIP index (inner product on normalised vectors equals cosine similarity).

The weakness of a bi-encoder is that it cannot compare the query and a candidate directly. It reliably identifies passages on the same *topic*, but can miss the one word that changes the legal meaning. For instance, two chunks describing different members' salaries will have nearly identical embeddings, even though only one answers the specific query.

2.2 Cross-Encoder (Pipeline 2, Re-Ranking)

A cross-encoder concatenates the query and a candidate chunk into a single input and runs a joint self-attention pass over both. This lets it compare wording directly and attend to phrases such as 'not entitled', 'except', and specific role names simultaneously. The relevance score is therefore much better calibrated to the actual question. I used **ms-marco-MiniLM-L-6-v2**, a lightweight model fine-tuned on passage re-ranking. Because the cross-encoder cannot reuse stored document vectors, I apply it only to the 10 candidates the bi-encoder already selected, keeping the latency increase bounded.

Component	Pipeline 1 (Baseline)	Pipeline 2 (Re-Ranked)
First-stage retriever	all-MiniLM-L6-v2 bi-encoder	Same
Vector index	FAISS IndexFlatIP (cosine)	Same
Candidates retrieved	Top 10	Same top 10
Context to LLM	Top 3 by bi-encoder score	Top 3 by cross-encoder score
Re-ranking model	None	ms-marco-MiniLM-L-6-v2
Answer model	gemini-3.7-flash, temp 0	Same

Table 2. System configuration for both pipelines.

3. Results: Retrieval Quality and Answer Impact

Table 3 summarises the gold-chunk retrieval performance across all 112 queries. I focus on **Hit@3** because the LLM sees only three chunks; getting the gold evidence into that context window is the necessary condition for a correct answer. Re-ranking improved Hit@3 by 7.1 percentage points and MRR@10 by 6.0 points. Hit@10 is unchanged because the cross-encoder only reorders the initial ten candidates and cannot recover gold chunks that the bi-encoder missed entirely.

Metric	Baseline (P1)	Re-Ranked (P2)	Change
Hit@1	0.393	0.446	+0.053
Hit@3	0.518	0.589	+0.071
Hit@10	0.696	0.696	0.000
MRR@10	0.473	0.533	+0.060

Table 3. Gold-chunk retrieval performance across 112 stress-test queries.

Table 4 breaks down what happened to the gold chunk for each query after re-ranking. The most important row is 'Promoted into top 3': these 11 queries are where re-ranking directly enabled a better answer. The 34 queries in 'Not in the initial top 10' expose the ceiling of the two-stage design. No amount of re-ranking can help if the first-stage retriever did not include the gold chunk in its candidate list.

Outcome after re-ranking	Count
Already in baseline top 3	55
Promoted into top 3 (fixed)	11
Not in the initial top 10	34
Still outside top 3	9
Moved down (worse)	3
Total	112

Table 4. What happened to the gold chunk after re-ranking.

3.1 Representative Rank Shifts

Table 5 shows seven queries selected for deep inspection: six where re-ranking rescued the gold chunk, and one (EXC-20) where it made things worse. The clearest case is DIS-05, where a single rank difference caused the answer to change from an incorrect salary (Rs 4,00,000) to the correct one (Rs 4,50,000).

Case	Type	Query (abbreviated)	Rank shift	Answer outcome
DIS-15	Distractor	Penalty for breaking voluntary undertaking?	7 to 1	Omitted, then correct
DIS-05	Distractor	Monthly salary of the Chairperson?	6 to 2	Wrong (4,00,000), then correct
NEG-10	Negation	Are Board Members paid a sitting fee?	5 to 1	Omitted, then correct
NEG-06	Negation	Can a Consent Manager read the data it routes?	5 to 3	No answer, then correct
DIS-18	Distractor	Who selects the Chairperson?	9 to 3	Omitted, then correct
EXC-63	Exception	Are courts exempt from the Act's duties?	4 to 2	Omitted, then correct
EXC-20	Exception	Indian firm processing foreign customers' data?	1 to 7	Correct, then omitted (worse)

Table 5. Rank of the gold chunk before and after re-ranking, with observed answer impact.

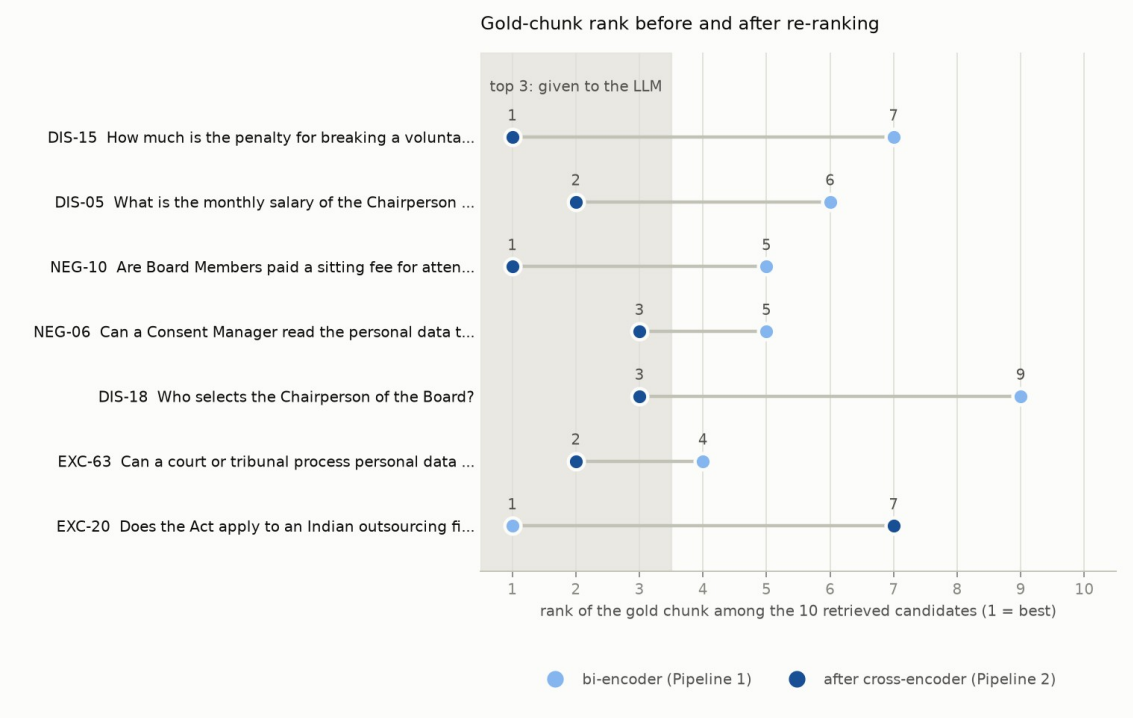


Figure 1. Rank of the gold chunk before and after re-ranking. Ranks 1 to 3 (shaded) are the slots visible to the LLM.

4. Latency Cost

Latency was measured on CPU over 180 timed runs (six rescued queries, each repeated 30 times after a warm-up pass). The Gemini API call is network-bound and excluded from the retrieval figures; it is reported as a pooled median separately.

Stage	Median (ms)	p95 (ms)
Query embedding (bi-encoder)	12.50	14.26
FAISS search (top 10)	0.07	0.12
Pipeline 1 retrieval total	12.58	14.40
Cross-encoder scoring (10 pairs)	361.46	384.00
Pipeline 2 retrieval total	374.40	395.89
Gemini API call (pooled median)	7,239	n/a

Table 6. Retrieval latency, CPU, 180 timed runs. Gemini call is a pooled median from saved responses.

The cross-encoder accounts for almost all of the extra 361.8 ms, roughly 30 times the cost of the baseline retrieval stage. In this setup the Gemini call dominates end-to-end time at roughly 7.24 seconds, so the retrieval difference is imperceptible to a human user. In a high-throughput system or with a faster local answer model, the retrieval cost would become the bottleneck.

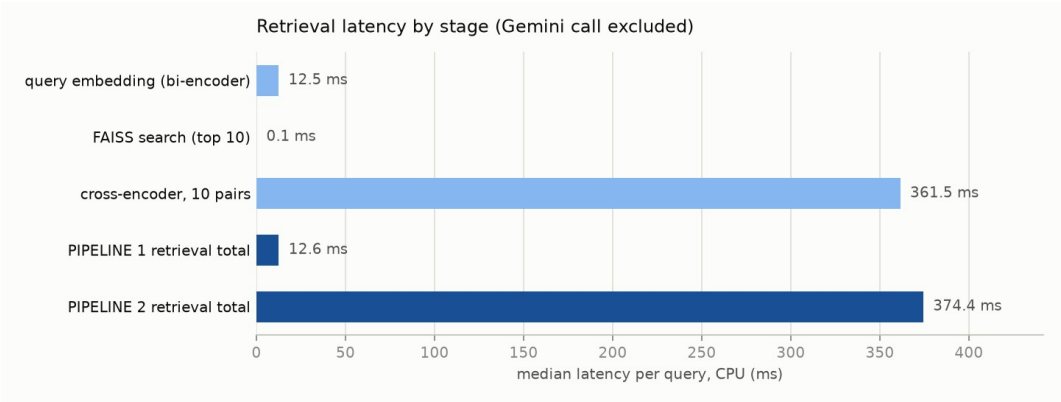


Figure 2. Median retrieval latency by stage. Gemini generation time is excluded.

5. Scaling and Conclusion

Because a cross-encoder must run a fresh forward pass for every (query, chunk) pair, exhaustive re-ranking does not scale. On this CPU, scoring all 230 chunks took 9.75 seconds (roughly 42.4 ms per pair). Table 7 extrapolates that cost to larger collections.

Collection size	Estimated time per query
230 chunks (this experiment)	9.8 s
10,000 chunks	about 424 s (7.1 min)
100,000 chunks	about 4,240 s (70.7 min)
1,000,000 chunks	about 42,404 s (11.8 hours)

Table 7. Estimated exhaustive cross-encoder time per query (42.4 ms/pair).

By contrast, a FAISS exact search over 10,000 vectors takes 1.78 ms and over 100,000 vectors just 17.57 ms. This is why the two-stage design is the standard: the bi-encoder provides fast, scalable recall over the full collection, and the cross-encoder spends its compute budget only on a short candidate list. The remaining weakness is clear: 34 of the 112 queries had their gold chunk outside the initial top 10, so re-ranking had no opportunity to recover them.

Cross-encoder re-ranking improved Hit@3 by 7.1 percentage points and corrected concrete answer failures including a wrong salary figure, missing prohibitions, and overlooked legal exceptions. The cost was real: median retrieval latency rose from 12.58 ms to 374.40 ms, and three queries were made worse. Re-ranking is best understood as a *precision improvement within a fixed recall budget*, not a recall improvement. Improving first-stage recall, for instance through hybrid sparse-dense retrieval or a larger candidate pool, is likely to have a greater long-term impact than refining the re-ranking model alone.